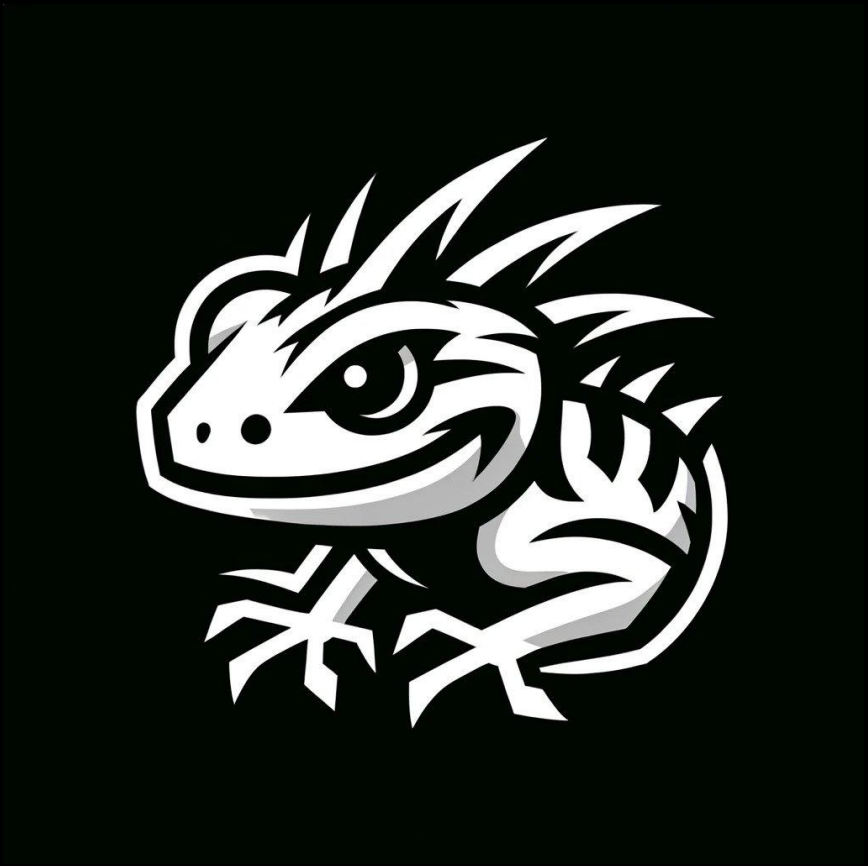
Gecko Fuzz



Autonomous On-Chain Fuzzing on XRP (EVM)

Technical Architecture Documentation

Jeevan Jutla

ripple

Gecko Fuzz is a novel smart contract security tool that leverages formal verification (concolic execution) assisted fuzzing algorithms guided by dataflow patterns and comparisons to audit smart contracts on-chain with no manual effort. You don't need to write invariants or manually specify the input generation strategy. Users supply contract addresses or native compiled contracts and Gecko autonomously generates the exploit when vulnerabilities are found. It can be easily integrated into CI/CD pipelines for a continuous innovation service and is highly configurable and can test different aspects of the smart contracts.

Contents

1. Overview **3**
2. Introduction **4**
3. Motivating Example **6**
3.1 Example: Transaction Sequence Exploit 6
4. Technical Architecture **8**
4.1 Gecko Fuzz Architecture 8
4.2 Capture Based Fuzzing 9
4.3 Dataflow Waypoint 10
4.4 Comparison Waypoint 11

1. Overview

A Gecko is a small, mostly carnivorous lizard known for feeding off bugs. Similarly, Gecko Fuzz is a novel autonomous on-chain smart contract auditing tool combining fuzzing and formal verification to find bugs in your code.

The importance of smart contract auditing has surged with the rise of Web 3.0 technology, becoming a billion-dollar industry. This growth is driven by the increase in the number of cyber-attacks and the widespread adoption of blockchain technology. Typically, smart contracts are audited using fuzzing tools. However, existing tools are limited to writing long and complex tests in local deployment environments and don't support auditing directly on the blockchain. This imitation is significant because blockchain states are constantly changing and vulnerabilities can be exploited at any moment. Fuzzing directly on the blockchain is challenging due to the persistent nature of blockchain states and the complexity of transaction sequences, which leads to a suboptimal exploration for both input and program spaces. Current fuzzers struggle with efficiently fetching blockchain states and completing audits promptly.

On-chain auditing or performing fuzz testing using real-time blockchain states is crucial for two reasons. First, certain code paths in smart contracts are only triggered by specific on-chain states, rendering local testing ineffective. Second, modern smart contracts often rely on external contracts for information necessitating real-time data retrieval from the blockchain for thorough auditing.

Unlike traditional fuzzers that rely on mutating transaction sequences, Gecko Fuzz takes captures of blockchain states and individual transactions. It introduces a dataflow waypoint mechanism to identify states with higher potential for vulnerabilities and uses comparison waypoints to streamline the state exploration process. By maintaining these captures Gecko Fuzz can quickly synthesise concrete exploits, such as re-entrancy attacks. Its ability to test smart contracts quickly makes it suitable for on-chain testing, offering numerous advantages over local deployment testing.

2. Introduction

Despite the benefits, on-chain auditing loses effectiveness if fuzzers cannot provide real-time responses. The primary goal is to stop a contract before attackers can exploit the vulnerabilities. Existing fuzzers generally suffer from slow response times. Achieving comprehensive coverage of smart contracts takes significant time with current fuzzers, often days, whereas on-chain auditing demands responses within seconds. This technical architecture document introduces a novel capture-based fuzzing technique and fuzzer named Gecko Fuzz, capable of achieving high coverage over the code and states of smart contracts in just seconds, thus facilitating on-chain auditing for numerous contracts.

Smart contracts, although typically smaller in code size compared to complex software like browsers and operating systems, are challenging to fuzz due to their stateful nature and intricate dependencies with other contracts. Previous approaches such as cargo-fuzz have started each fuzz run from a fresh state, using a sequence of transactions as input and mutating parts of this sequence during the mutation phase. This results in high overhead from re-executing transactions to return to a previous state, and the cost increases linearly for deeper states that require multiple transactions to build. Furthermore, existing tools lack feedback mechanisms for states, focusing only on transactions, despite different explorations challenges for stats and transactions. We argue that the 'interestingness' of states is just as crucial as transactions in stateful fuzzing and current tools lack mechanisms to identify and explore these interesting states.

Our solution, capture-based fuzzing avoids the need to re-execute inputs to rebuild previous states. Capturess are replicas of intermediate states created from some transactions. By storing all relevant capturess in a state corpus, Gecko can 'time travel' to previous states with constant complexity (O(1)), enabling efficient exploration of both transactions and states. However, storing all captures in the corpus isn't feasible due to limited runtime memory resources, as the size of the captures can grow to several gigabytes within seconds. To address this, we developed two feedback mechanisms (waypoints) to identify interesting states and a corpus pruning technique to manage the number of states when necessary.

By reducing re-execution overhead and enhancing the feedback mechanism, we significantly decrease fuzzing time, allowing most vulnerabilities to be identified instantly. Thus, Gecko can support on-chain auditing objectives, which require swift response times to pre-empt attackers while requiring no manual specifications of input generation strategies or invariants.

Main Contributions of Gecko Fuzz:

- A novel capture-based fuzzing algorithm to minimise re-execution overhead in stateful smart contract fuzzing.
- New waypoint mechanisms optimised for prioritising exploration of interesting capture states, enabling efficient program exploration:
    - The dataflow waypoint assesses state interestingness based on future memory loads.
    - The comparison waypoint manages the state corpus size through probabilistic sampling and hard comparison feedback.
- The development of Gecko, a fast and efficient smart contract fuzzer, proved effective in practice.
- A new autonomous method for smart contract auditing using real-time state data from the blockchain, demonstrating its capability to detect and reproduce exploits in on-chain projects worth millions.

3. Motivating Example

3.1 Example: Transaction Sequence Exploit

To see why current sequence-based fuzzers fail to exploit some vulnerabilities, consider a simple yet realistic smart contract program in Fig 1, with *s* a single state variable *counter*. The function *incr* increases the *counter* by one when the argument is smaller than the current *counter* and similarly, *decr* decreases the *counter* when the argument is greater than *counter.* Function *buggy* introduces a bug when the counter reaches some constant *T*.

The program accepts sequences of transactions as inputs. Each transaction is a pair of a function call and its parameters. For instance if T==2, a sequence that would trigger the bug is [(incr,0), (incr,1), (buggy)]. This also means the 'incr' function is called twice with valid arguments, raising the *counter* to 2, followed by a call to *buggy*, which then triggers the bug.

Although the above input sequence seems simple and easy to synthesise, constructing a more complete transaction sequence when *T* is large is non-trivial. The bug triggering transaction sequence can be very long and complex in real-world scenarios. For instance, hackers leveraged six transactions, each with on average 40 function calls to build up the state that makes Team Finance decentralised finance (DeFi) platform vulnerable to attack (https://www.cryptopolitan.com/team-finance-exploited-for-15-million/).

When exploring the transaction sequence space of this simple smart contract, using cargo-fuzz or echidna, it fails to quickly detect a bug as *T* increases. This is because when evaluating a transaction sequence, cargo-fuzz and echidna need to re-execute the entire sequence of transactions from the very beginning, including deploying the transaction to be tested. Time spent executing a transaction in an arbitrary state needs to include the time spent re-executing the sequence of transactions required to reach the arbitrary state.

The amortised re-execution time grows exponentially as the length of the sequence of transactions grows linearly. The re-execution time can be eliminated if the state is reached after executing a sequence of transactions can be memoized. However memoization requires saving the number of states that is exponential in the number of transaction sequences. The key insight in Gecko Fuzz is that we can only memoize a set of interesting states called captures, instead of memoizing all intermediate states and using these interesting states to only explore new states without re-execution. The interestingness of a state is defined using two novel waypoint ideas.

```
contract SimpleState {
    int256 counter = 0;

    function incr(int256 x) public {
        require(x <= counter);
        counter += 1;
    }

    function decr(int256 x) public {
        require(x >= counter);
        counter -= 1;
    }

    function buggy() public {
        if (counter == T) {
            bug!();
        }
    }
}
```

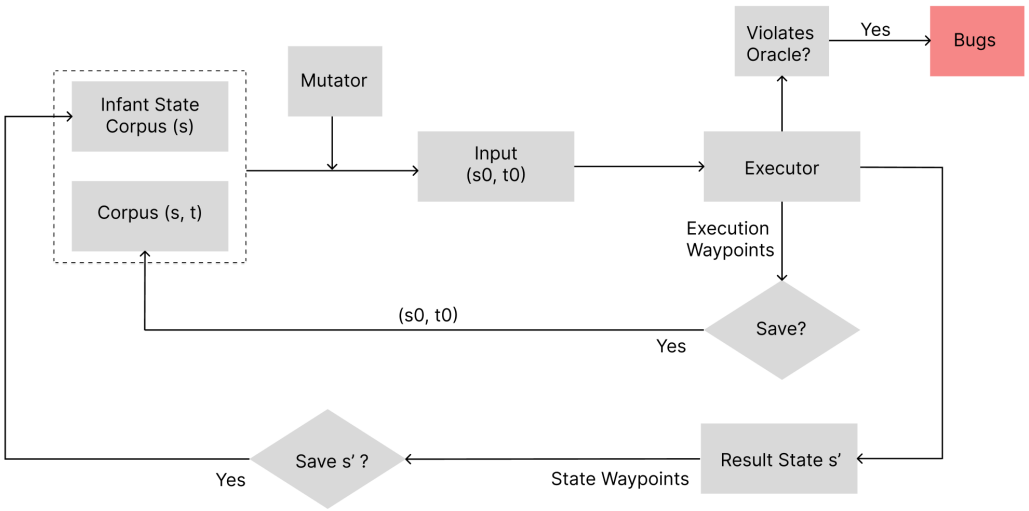
Figure 1: Smart contract with a simple persistent state counter.

4. Technical Architecture

This section introduces an overview methodology to develop Gecko Fuzz. To view the code go to https://github.com/jjjutla/Gecko-Fuzz.

4.1 Gecko Fuzz Architecture

An overview of the Gecko Fuzz Architecture showing the interaction of the state waypoints and captures of interesting result states:



Algorithm 1: Gecko Fuzz Fuzzing Algorithm:

```
C ← Corpus() # transaction and state pair corpus
Cs ← Corpus() # infant state corpus

while true:
    (s,t) ← Next(C)
       if Random(0, 1) > P:
          tmut ← Mutation(t) # mutate transaction
          smut ← s
       else
          smut ← Next(Cs ) # mutate state by fetching from infant
state corpus
          tmut ← t

# execute transaction tmut on state smut
# f represents waypoints, s' is the resulting state

f , s' ←Executor(smut, tmut)
if W(s,t ) (f) is interesting: # get feedback for waypoints of
execution
    C ← C ∪ (smut, tmut)
if W(s) (f, s') is interesting: # get feedback for waypoints of
infant state corpus
```

$C_s \leftarrow C_s \cup s'$

4.2 Capture Based Fuzzing

In smart contract executions, a state *s* can be constructed by executing a sequence of transactions from an initial state. To travel back to some intermediate state, a common practice for existing stateful fuzzers is to re-execute the transactions to construct the specific state from the initial state. Instead of re-executing the previous transactions, we directly capture the state and save the unique states. We maintain a corpus that stores pairs of the (s,t), where $s \in S$ and $t \in T$. Specifically, given an execution Execution$(s,t) \mapsto \rightarrow s'$, where $s' \in S$ is the new state after execution of t on s, the pair (s, t) is added to the corpus when the combination of feed- back (waypoints) for C reported by the execution of the transaction on that state is interesting. Multiple waypoint mechanisms, including coverage-guided feedback, dataflow waypoints and comparison waypoint as well as research into techniques for using LLMs to guide and prioritise fuzzing of smart contracts intelligently.

Since Gecko Fuzz adds the states before a transaction to the corpus *C* the state obtained after executing the transaction is lost. Therefore if Gecko keeps mutating the states from the corpus it is only going to explore the initial state with random transactions. The subsequent states built with transactions on the initial states are never explored, undermining the purpose of stateful fuzzing, which is to explore all potential states. To make exploration more efficient and to avoid re-execution, Gecko maintains a separate corpus of states to memoize the states after execution which we call the 'infant state corpus', C_s. Specifically given an execution the state s' is added to the infant state corpus when Gecko Fuzz finds it interesting using a combination of waypoints. Note that the waypoints for the infant state corpus are different from that of *C* - they determine whether the current execution is interesting, but waypoints for the infant state corpus determine whether such a state can lead to future interesting executions.

The algorithm in Algorithm 1 summarises Gecko's capture based fuzzing algorithm. During each iteration of the fuzz loop, the fuzzer first picks a state and transaction pair from the corpus *C*. The fuzzer then either mutates the transaction *t* structurally to $t$mut or replaces the states *s* with a state *smut*, stored in the infant states corpus C_s. The new mutant is always reachable in the input space because the selected state from the infant state corpus can be constructed by a sequence of transactions, and the state allows the execution of a new transaction. The mutant is executed, which then yields a new state. After execution Gecko Fuzz saves the state and transaction pair (smut, $t$mut) to corpus *C* if the observed waypoints are interesting. Gecko adds the resulting state s' to the infant state corpus C_s if s' may lead to a potentially interesting executions with new transactions. The dataflow waypoint and comparison waypoint are discussed to decide if s' is interesting.

4.3 Dataflow Waypoint

Defining waypoints for measuring the interestingness of an input during execution has been explored in https://dl.acm.org/doi/10.1145/3360600. Yet there has been no work on how to define waypoints for states. The goal for the waypoints is to save resulting states in the corpus only if future executions from these states are interesting. Therefore, testing if a state is interesting also requires semantic information about the state in order to capture 'future interestingness'.

We introduce two state waypoints for dataflow analysis, focusing on dynamic and static aspects. The dynamic 'dataflow waypoint' identifies memory locations as interesting if they appear in future load instructions, using instrumentation to monitor these instructions in real-time. This contrasts with the conventional static analysis, which is less effective for smart contracts that dynamically interact with external contracts. Our approach involves approximating future 'interesting' load locations based on past loads. A memory location is deemed interesting if it has been written with unique values in the past, suggesting future writes as well. This is tracked by Gecko, which monitors past loaded memory locations and their abstract values. A store instruction is considered interesting if it writes a new abstract value to a previously interesting location. We employ two algorithms and maintain two maps to record past abstract memory locations of load and store instructions. The load map marks a memory location as interesting upon a load instruction, while the store map tracks abstract value buckets, a concept borrowed from AFL's bucketing mechanism. A state change is deemed interesting if a store instruction alters a bucket value to true, matching a true value in the corresponding load map location. This method, while increasing the number of interesting states and potential storage overhead, allows for a more nuanced analysis of state changes.

Algorithm 2: For dataflow waypoint instrumentation:

```
Result: L, S
while is Executing do
 if Current Instruction is load(Loc) then
   L (Loc % MAP_SIZE) ← true ;
 if Current Instruction is store(Loc, Value) then
   (Loc % MAP_SIZE)(BucketOf(Value)) ← true;
```

Algorithm 3: For dataflow waypoint evaluation:

```
Data: L, S
Result: IsStateInteresting
/* Interesting if the bucket changes from false to true */
if ∃Bucket ∈ S s.t. Bucket changes from false to true then
 /* Interesting if the location has been loaded before */
 if L at the index of Bucket then
   IsStateInteresting = true;
```

4.4 Comparison Waypoint

Comparison waypoints focus on intermediate states where comparison operands gradually converge in successive executions, aiding in achieving higher coverage. For instance in Figure 1, comparison waypoints find states where the counter incrementally increases from 0 to 10 as relevant, because the counter approaches the comparison target on line 16. The proposed algorithm started with initialising a local map with a maximum value. During execution Gecko Fuzz updates the distance at each comparison instruction, reflecting how close the operands are to being equal. An execution is considered interesting if it brings any comparison closer to being true than before. For memory efficiency Gecko Fuzz prunes less interesting states from the infant state corpus based on comparison waypoints. A voting algorithm determines a state's interestingness each time and execution on an initial state reduces the comparison map distance, that state gains a vote. To address the issues of a rapidly growing infant state corpus with dataflow waypoints, Gecko Fuzz prioritises states with more votes for exploration and employs a probabilistic sampling algorithm, alternating between random and probabilistic sampling. Additionally, Gecko Fuzz prunes the least interesting and most explored states when the corpus size exceeds a threshold, discarding states with the lowest votes/visits ratio that have been explored beyond a set limit. This pruning strategy is resource-intensive but infrequent, activated only when the corpus size hits the threshold.

Algorithm 4: For comparison waypoint instrumentation

```
Result:CLocal
CLocal ← Vec<u256> - U256::max
 while is Executing do
  if Current Instruction is lt(Lhs, Rhs) then
  CLocal(PC % MAP_Size)← Lhs - Rhs;
  if Current Instruction is eq(Lhs, Rhs) then
  CLocal(PC % MAP_Size)← abs(Lhs - Rhs);
```

Algorithm 5: For comparison waypoint evaluation

```
Data:C,CLocal,s
 for c ∈ C, cLocal ∈ CLocal do
  if c > cLocal then
   s.votes = s.votes + 1
```